Used Car Price Predictor

Machine Learning Project with a Web Interface (Quikr Dataset)

1. Project Overview

Used Car Price Predictor estimates the resale price of a used car from five details: company, model, year of purchase, fuel type and kilometres driven. The project covers the full machine learning workflow: raw marketplace data is cleaned, a Linear Regression model is trained, and the saved model is served through a simple web page where a user enters the car details and instantly sees an estimated price in rupees.

2. Problem Statement

People who buy or sell a used car often do not know a fair price. Listings on classified sites such as Quikr are inconsistent: prices are typed in many formats, some say "Ask For Price", and several fields contain spam or non-numeric values. A model trained on cleaned listing data can give a quick, data-based price estimate, but only if the data is cleaned properly and the model is easy to use. This project addresses both parts.

3. Project Objectives

- Clean the raw Quikr car listings and make them suitable for modelling.
- Study how company, year, kilometres driven and fuel type relate to price.
- Train a regression model that predicts the price of a used car.
- Evaluate the model honestly on data it has not seen.
- Provide a simple web interface so a non-technical user can get a prediction.

4. Dataset and Data Cleaning

The dataset is the public Quikr used-car listing file with 892 rows and six columns: name, company, year, Price, kms_driven and fuel_type. All columns arrive as text.

Problem in raw data	Cleaning step	Result
Non-numeric values in year	Keep only numeric years, convert to integer	Year usable as a number
Price contains "Ask For Price"	Remove those rows	Only real prices remain
Price has commas and is text	Remove commas, convert to integer	Price usable as a number
kms_driven ends with "kms" and has commas	Keep the number part, remove commas, convert to integer	Distance usable as a number
Missing or invalid kms and fuel type	Remove those rows	No missing values
Very long, inconsistent car names	Keep only the first three words	Fewer, consistent model names
One extreme outlier price	Remove prices of Rs.60,00,000 and above	Less distortion in training

After cleaning, **815 of the 892 rows** remain.

5. Model Design

Component	Role	Output
Input features	name, company, year, kms_driven, fuel_type	One row per car
One-hot encoder	Converts name, company and fuel type into 0/1 columns; unseen categories are ignored	Numeric feature matrix
Pass-through	year and kms_driven are used as they are	Numeric columns
Linear Regression	Learns a weight for every feature	Predicted price in rupees

The encoder and the regression are combined in a single scikit-learn Pipeline, so the same preprocessing is applied automatically at training time and at prediction time. The data was split 80% for training and 20% for testing with a fixed random seed.

6. System Architecture

1. **Raw CSV** (data/quikr_car.csv) is read by train.py.
2. **train.py** cleans the data, trains the pipeline and saves LinearRegressionModel.pkl and options.json.
3. **Flask server** (app.py) loads the saved model and the dropdown options.
4. **Web page** (templates/index.html) shows the form and sends the entered values to the /predict endpoint.
5. **/predict** builds a one-row table, runs the pipeline and returns the price.

7. Workflow

1. Load the raw listings.
2. Clean and convert every column.
3. Select the features (name, company, year, kms_driven, fuel_type) and the target (Price).
4. Split the data into training and test sets.
5. Train the pipeline and measure R^2 on the test set.
6. Refit on all cleaned data and save the model with pickle.
7. In the web page, the user picks a company; the model list updates to that company's cars.
8. The user enters year, fuel type and kilometres, then clicks **Predict price**.
9. The server returns the estimate, which the page displays.

8. Technology Stack

Layer	Tools
Language	Python 3
Data handling	pandas
Machine learning	scikit-learn (OneHotEncoder, LinearRegression, Pipeline)
Model storage	pickle (LinearRegressionModel.pkl)

Layer	Tools
Backend	Flask
Frontend	HTML, CSS and JavaScript with light and dark themes
Development	VS Code, run locally

9. Evaluation

The model is scored with R^2, which measures how much of the variation in price the model explains (1.0 is perfect).

Check	Result
Single 80/20 split (fixed seed 42)	$R^2 = 0.573$
5-fold cross-validation	R^2 between 0.47 and 0.78
200 different random splits	R^2 from about -0.22 to 0.86, median about 0.63

Important note on the original notebook. The original notebook reported an R^2 of about 0.92. That figure was obtained by trying 1000 random train/test splits and keeping the best one, which overstates how well the model will do on new cars. This project reports the fixed-split and cross-validated scores instead, which are more realistic.

Interface check. The web endpoints were tested with sample inputs:

Car	Year	Kms	Fuel	Predicted price
Maruti Suzuki Swift	2019	100	Petrol	Rs.4,04,721
Hyundai i20 Sportz	2014	50,000	Diesel	Rs.3,45,917

10. Screenshots of the Application

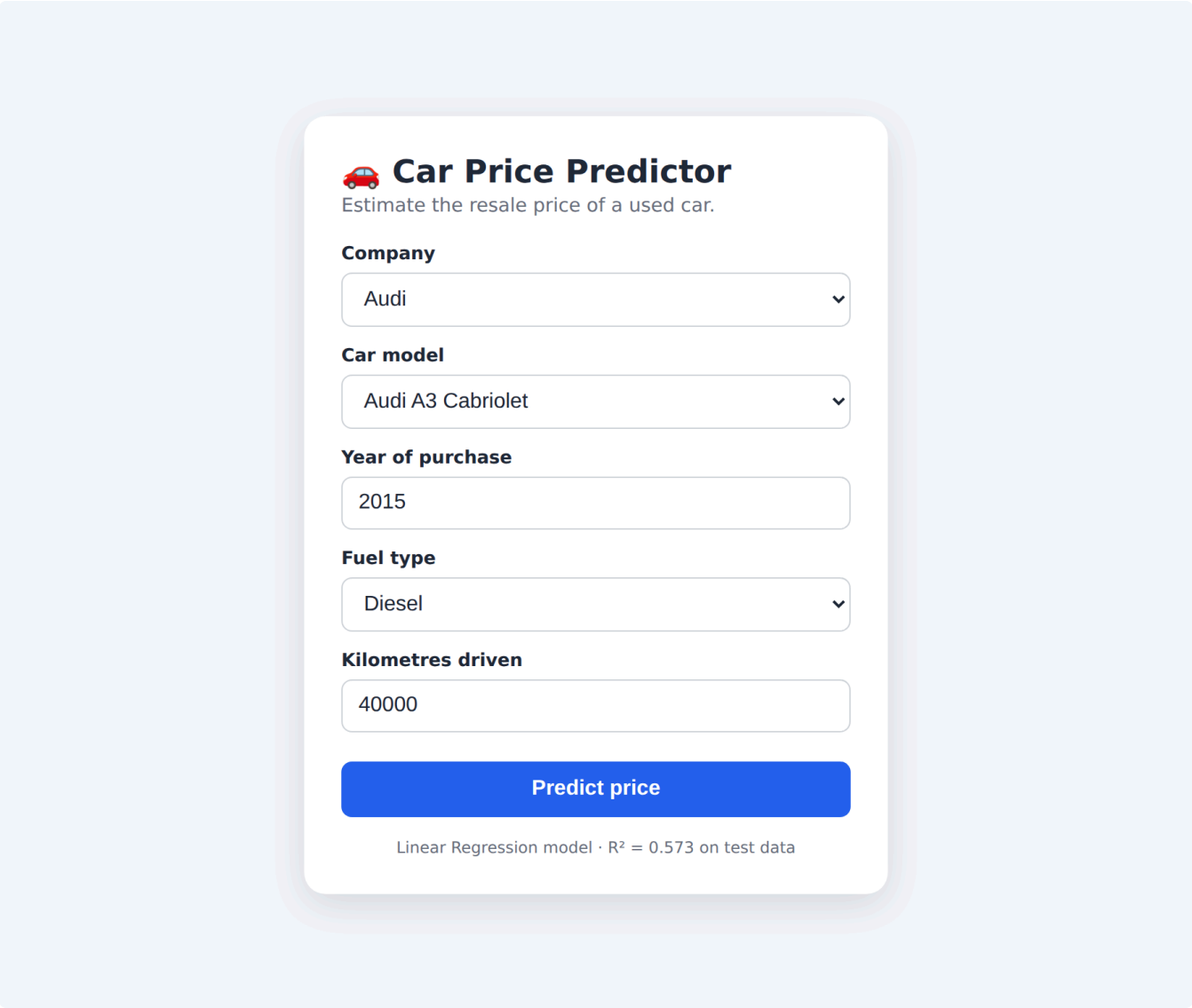


Figure 1. Prediction page with the form (company, model, year, fuel type, kilometres driven).

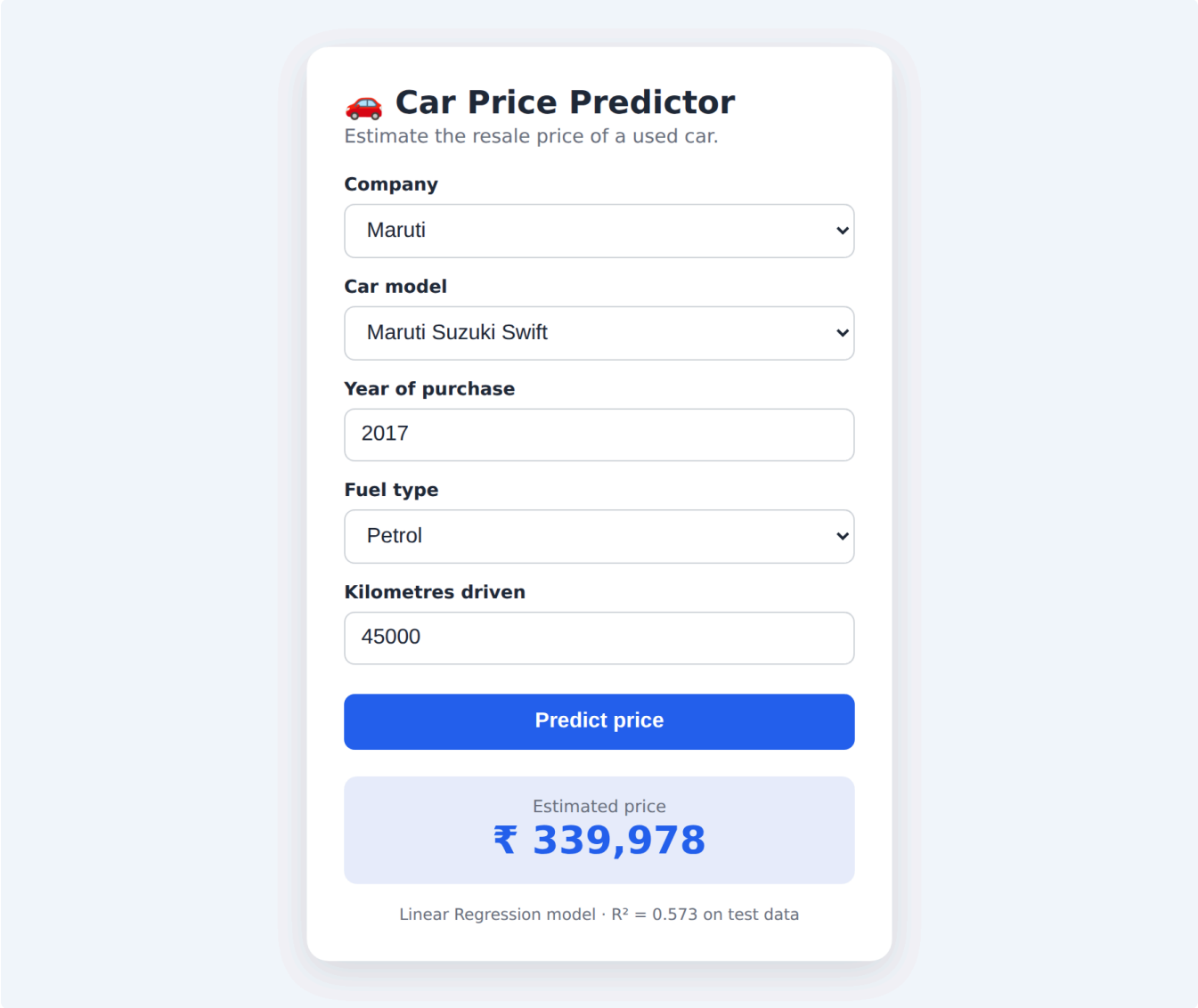


Figure 2. Result shown after clicking Predict price.

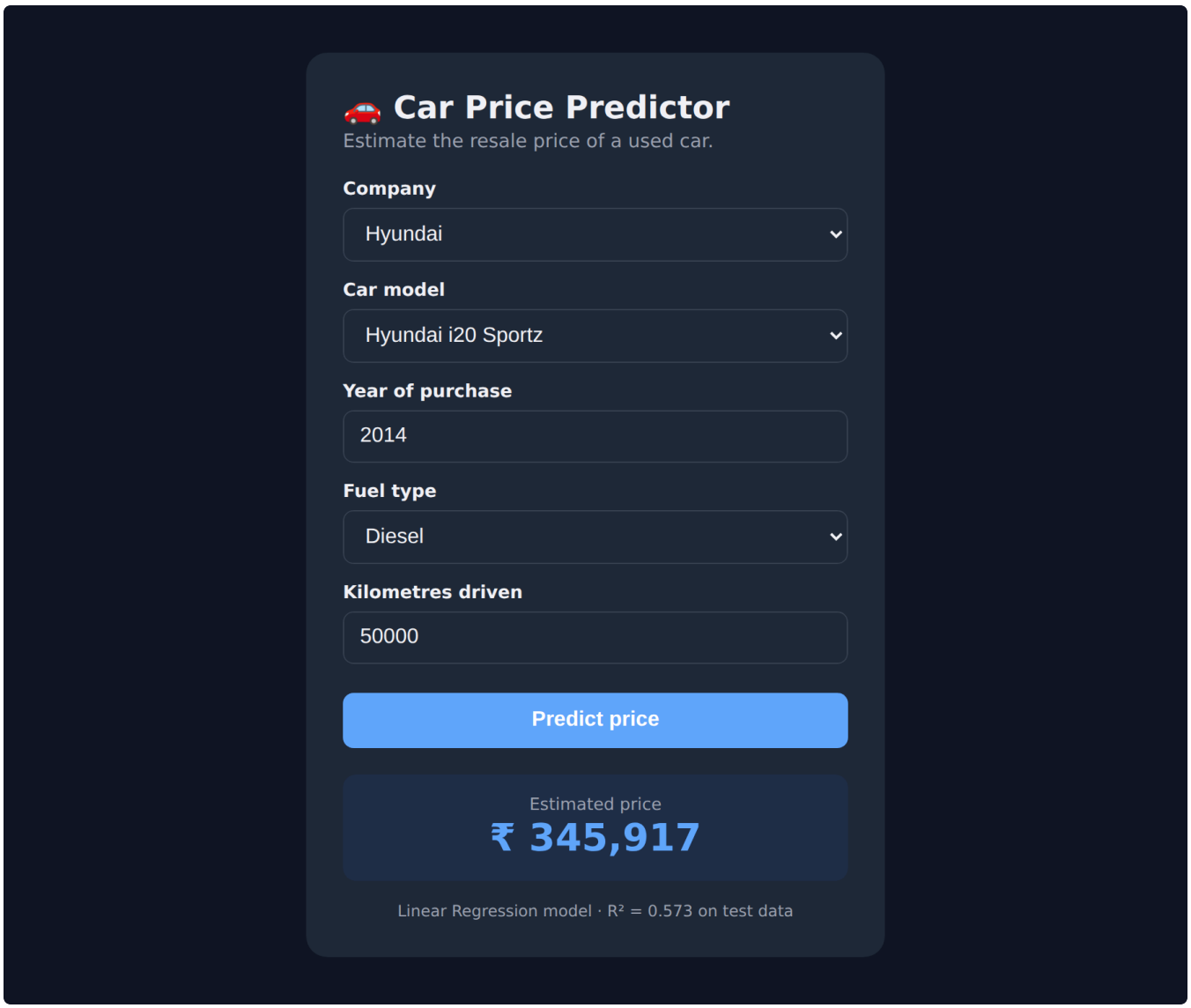


Figure 3. Page in dark mode.

11. Expected Outcome and Limitations

The outcome is a working prototype: data is cleaned, a model is trained and saved, and a user can get a price estimate from a web page. It is a rough guide, not a valuation.

- Linear Regression is a simple model, and accuracy varies considerably between data splits.
- The dataset is small (815 cleaned rows), so rare car models have few examples.
- Prices come from asking prices on a listing site, not final sale prices.
- Condition, accident history, location and trim details are not in the data.
- The model only knows the companies and models present in the dataset.

- The app runs locally and is not deployed or containerized.

12. Future Work

- Try Random Forest or Gradient Boosting and compare them using cross-validation.
- Predict the logarithm of the price to handle the wide price range.
- Show a price range instead of a single number.
- Add more data and extra features such as owner count and transmission.
- Deploy the app online with gunicorn on a free hosting service.

13. Conclusion

The project shows the complete path from messy raw data to a usable application: cleaning, feature encoding, model training, honest evaluation and a web interface. The honest score is lower than the notebook's original figure, which highlights why evaluating on unseen data matters, and it points to clear next steps for improving the model.